

Running Notes-22. LangChain & Agentic AI – From Basics to Advanced 22

LangChain Runnable — stream()

1. The Third Operation — stream()

Consider ChatGPT itself.

When you ask a question, sometimes the answer appears gradually.

You don't always have to stare at a blank screen until the entire answer is complete.

This experience is called:

STREAMING

2. What Is Streaming?

Streaming means receiving output in pieces as it becomes available instead of waiting for the complete final output.

Normal Execution

Question

↓

Wait

↓

Wait

↓

Complete Answer

Streaming Execution

Question

↓

Part 1

↓

Part 2

↓

Part 3

↓

...

3. What Is stream()?

LangChain Runnables define:

stream()

for streaming output from a **single input** as it is produced when the Runnable/components support streaming.

Simple Formula

ONE INPUT

↓

stream()

↓

OUTPUT CHUNKS

↓

CHUNK

CHUNK

CHUNK

...

4. What Is a Chunk?

During streaming, we often call each piece:
chunk

A **chunk** is simply a piece of streamed output.

Very Important

A chunk does **not necessarily mean exactly one English word**.

5. Simple stream() Program

import os

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
```

```
prompt = PromptTemplate.from_template(
    """
    Explain {topic} in simple English
    in about 5 short points.
    """
)
```

```
model = ChatOpenAI(  
    model="gpt-5.6-luna",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

```
parser = StrOutputParser()
```

```
chain = prompt | model | parser
```

```
s = chain.stream(  
    {"topic": "Artificial Intelligence"}  
)
```

```
print(type(s)) # generator object
```

```
for chunk in s:  
    # print(chunk)  
    pass
```

Important Observation

```
s = chain.stream(...)
```

returns a stream/generator-like object that we can iterate over.

We process the output using:

```
for chunk in s:
```

6. `time.sleep()`

Python provides the time module.

```
import time
```

We can pause execution using:

```
time.sleep(seconds)
```

Example:

```
import time
```

```
print("Hello")
```

```
time.sleep(2)
```

```
print("Friends")
```

The program waits for approximately 2 seconds before printing:

Friends

7. Why `end=""`?

Normally:

```
print("Hello")
```

```
print("Friends")
```

Output:

Hello

Friends

Every `print()` normally moves to a new line.

But:

```
print("Hello ", end="")
```

```
print("Friends")
```

Output:

Hello Friends

Therefore, for streaming:

```
end=""
```

prevents Python from forcing every chunk onto a new line.

Example:

```
print(chunk, end="")
```

8. Why flush=True?

```
flush=True
```

asks Python to push the printed output immediately rather than holding it in an output buffer.

For streaming displays, this is commonly useful:

```
print(  
    chunk,  
    end="",  
    flush=True  
)
```

Meaning

chunk

↓

print immediately

↓

don't wait in output buffer

9. Complete Interactive Streaming Program

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import PromptTemplate
```

```
from langchain_core.output_parsers import StrOutputParser
```

```
prompt = PromptTemplate.from_template(
```

```
    """
```

```
    Explain {topic} in very simple English.
```

```
    Give:
```

```
    - Definition
```

```
    - 5 important points
```

```
    - One example
```

```
    - One-line conclusion
```

```
    """
```

```
)
```

```
model = ChatOpenAI(
```

```
model="gpt-5.6-luna",  
api_key=os.getenv("OPENAI_API_KEY")  
)
```

```
parser = StrOutputParser()
```

```
chain = prompt | model | parser
```

```
topic = input("Enter topic: ")
```

```
print("\nAI Response:\n")
```

```
for chunk in chain.stream(  
    {"topic": topic}  
):  
    print(  
        chunk,  
        end="",  
        flush=True  
    )
```

```
print()
```

Execution Flow

User enters topic

↓

PromptTemplate

↓

Chat Model

↓

Output Parser

↓

stream()

↓

Chunk

↓

Chunk

↓

Chunk

↓

Complete Response

10. Limitation of Streaming

stream() provides a streaming interface, but actual incremental streaming depends on the components involved.

For effective streaming, use:

Streaming-capable Model

+

Streaming-friendly Components

Important Note

If the components do not support streaming, then `stream()` may behave like `invoke()` by producing the result without true incremental streaming.

11. `invoke()` vs `stream()`

`invoke()`

```
result = chain.invoke(  
    {"topic": "Python"}  
)
```

```
print(result)
```

Concept

Input

↓

Wait for complete result

↓

Full Result

`stream()`

```
for chunk in chain.stream(  
    {"topic": "Python"}  
):  
    print(
```

```
    chunk,  
    end="",  
    flush=True  
)
```

Concept

Input

↓

Chunk

↓

Chunk

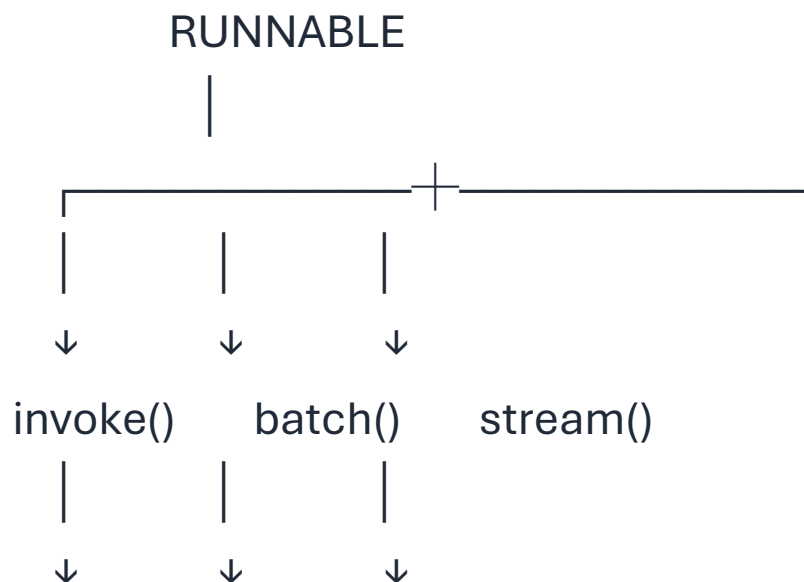
↓

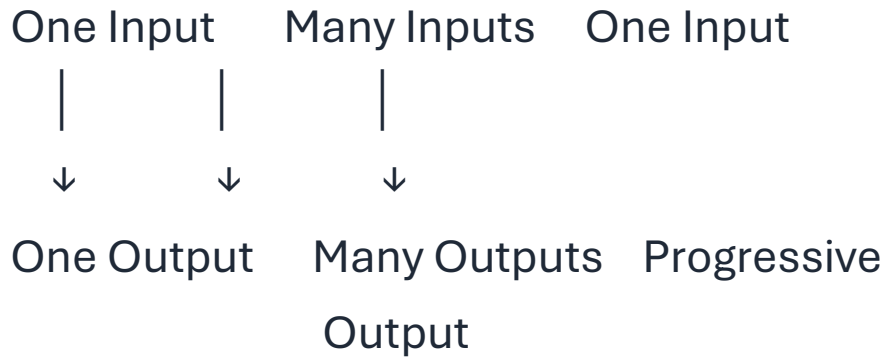
Chunk

↓

Complete

12. invoke() vs batch() vs stream()





Simple Memory Table

Method	Input	Output Style
invoke()	One Input	One Complete Output
batch()	Multiple Inputs	Multiple Outputs
stream()	One Input	Progressive Output / Chunks

13. Full Travel Planner Program

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import PromptTemplate
```

```
from langchain_core.output_parsers import StrOutputParser
```

```
prompt = PromptTemplate.from_template(
```

```
    """
```

```
    Create a {days}-day travel plan for {place}.
```

```
    Budget: {budget}
```

```
    Main Interest: {interest}
```

Keep the answer simple.

"""

)

```
model = ChatOpenAI(  
    model="gpt-5.6-luna",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

```
parser = StrOutputParser()
```

```
chain = prompt | model | parser
```

```
print("1. invoke")  
print("2. batch")  
print("3. stream")
```

```
choice = input("Enter choice: ")
```

```
if choice == "1":
```

```
    result = chain.invoke(  
        {
```

```
"days": 3,  
"place": "Goa",
```